

# Operating Systems

2520030042

K.Ashish

SEC-5

## WEEK-3

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

### Code

```
Ashish@Ashish: ~
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("\nChild process running.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        sleep(2);

        printf("Child process terminating.\n");
        exit(0);
    }
    else {
        printf("\nParent created child with PID: %d\n", pid);
        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Parent process terminating.\n");
    }

    return 0;
}
```

output

```
Ashish@Ashish:~$ nano fork.c
Ashish@Ashish:~$ gcc fork.c
Ashish@Ashish:~$ ./a.out
Parent process started.
Parent PID: 3758

Parent created child with PID: 3759
Parent is waiting for child...

Child process running.
Child PID: 3759
Child PPID: 3758
Child process terminating.
Parent process terminating.
Ashish@Ashish:~$ |
```